

✓ Welcome to Colab!

Explore the Gemini API

The Gemini API gives you access to Gemini models created by Google DeepMind. Gemini models are built from the ground up to be multimodal, so you can reason seamlessly across text, images, code and audio.

How to get started

- Go to [Google AI Studio](#) and log in with your Google Account.
- [Create an API key](#).
- Use a quickstart for [Python](#) or call the REST API using [curl](#).

Discover Gemini's advanced capabilities

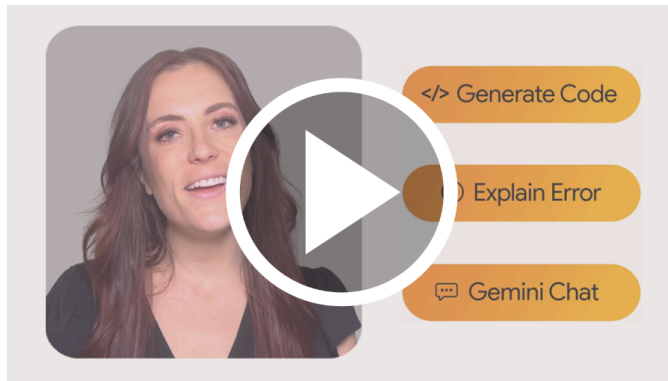
- Play with Gemini [multimodal outputs](#), mixing text and images in an iterative way.
- Discover the [multimodal Live API](#) (demo [here](#)).
- Learn how to [analyse images and detect items in your pictures](#) using Gemini (bonus, there's a [3D version](#) as well!).
- Unlock the power of the [Gemini thinking model](#), capable of solving complex tasks with its inner thoughts.

Explore complex use cases

- Use [Gemini grounding capabilities](#) to create a report on a company based on what the model can find on the Internet.
- Extract [invoices and form data from PDFs](#) in a structured way.
- Create [illustrations based on a whole book](#) using Gemini large context window and Imagen.

To learn more, take a look at the [Gemini cookbook](#) or visit the [Gemini API documentation](#).

Colab now has AI features powered by [Gemini](#). The video below provides information on how to use these features, whether you're new to Python or a seasoned veteran.



What is Colab?

Colab, or 'Colaboratory', allows you to write and execute Python in your browser, with

- Zero configuration required
- Access to GPUs free of charge
- Easy sharing

Whether you're a **student**, a **data scientist** or an **AI researcher**, Colab can make your work easier.

Watch [Introduction to Colab](#) or [Colab features you may have missed](#) to learn more or just get started below!

✓ Getting started

The document that you are reading is not a static web page, but an interactive environment called a **Colab notebook** that lets you write and execute code.

For example, here is a **code cell** with a short Python script that computes a value, stores it in a variable and prints the result:

```
seconds_in_a_day = 24 * 60 * 60
seconds_in_a_day
```

↩ 86400

To execute the code in the above cell, select it with a click and then either press the play button to the left of the code, or use the keyboard shortcut 'Command/Ctrl+Enter'. To edit the code, just click the cell and start editing.

Variables that you define in one cell can later be used in other cells:

```
seconds_in_a_week = 7 * seconds_in_a_day
seconds_in_a_week
```

↩ 604800

Colab notebooks allow you to combine **executable code** and **rich text** in a single document, along with **images**, **HTML**, **LaTeX** and more. When you create your own Colab notebooks, they are stored in your Google Drive account. You can easily share your Colab notebooks with co-workers or friends, allowing them to comment on your notebooks or even edit them. To find out more, see [Overview of Colab](#). To create a new Colab notebook you can use the File menu above, or use the following link: [Create a new Colab notebook](#).

Colab notebooks are Jupyter notebooks that are hosted by Colab. To find out more about the Jupyter project, see jupyter.org.

✓ Data science

With Colab you can harness the full power of popular Python libraries to analyse and visualise data. The code cell below uses **numpy** to generate some random data, and uses **matplotlib** to visualise it. To edit the code, just click the cell and start editing.

You can import your own data into Colab notebooks from your Google Drive account, including from spreadsheets, as well as from GitHub and many other sources. To find out more about importing data, and how Colab can be used for data science, see the links below under [Working with data](#).

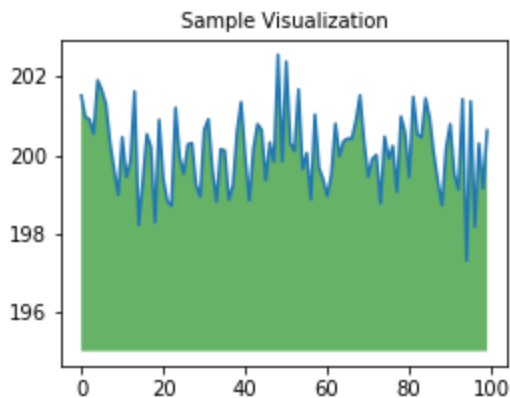
```
import numpy as np
import IPython.display as display
from matplotlib import pyplot as plt
import io
import base64

ys = 200 + np.random.randn(100)
x = [x for x in range(len(ys))]

fig = plt.figure(figsize=(4, 3), facecolor='w')
plt.plot(x, ys, '-')
plt.fill_between(x, ys, 195, where=(ys > 195), facecolor='g', alpha=0.6)
plt.title("Sample Visualization", fontsize=10)

data = io.BytesIO()
plt.savefig(data)
image = F"data:image/png;base64,{base64.b64encode(data.getvalue()).decode()}"
```

```
alt = "Sample Visualization"  
display.display(display.Markdown(F""""! [{alt}]({image})"""))  
plt.close(fig)
```



Colab notebooks execute code on Google's cloud servers, meaning that you can leverage the power of Google hardware, including [GPUs and TPUs](#), regardless of the power of your machine. All you need is a browser.

For example, if you find yourself waiting for **pandas** code to finish running and want to go faster, you can switch to a GPU runtime and use libraries like [RAPIDS cuDF](#) that provide zero-code-change acceleration.

To learn more about accelerating pandas on Colab, see the [10-minute guide](#) or [US stock market data analysis demo](#).

✓ Machine learning

With Colab you can import an image dataset, train an image classifier on it and evaluate the model, all in just [a few lines of code](#).

Colab is used extensively in the machine learning community with applications including:

- Getting started with TensorFlow
- Developing and training neural networks
- Experimenting with TPUs
- Disseminating AI research
- Creating tutorials

To see sample Colab notebooks that demonstrate machine learning applications, see the [machine learning examples](#) below.

✓ More resources

Working with notebooks in Colab

- [Overview of Colab](#)
- [Guide to markdown](#)
- [Importing libraries and installing dependencies](#)
- [Saving and loading notebooks in GitHub](#)
- [Interactive forms](#)
- [Interactive widgets](#)

Working with data

- [Loading data: Drive, Sheets and Google Cloud Storage](#)
- [Charts: visualising data](#)
- [Getting started with BigQuery](#)

Machine learning crash course

These are a few of the notebooks from Google's online machine learning course. See the [full course website](#) for more.

- [Intro to Pandas DataFrame](#)
- [Intro to RAPIDS cuDF to accelerate pandas](#)
- [Linear regression with tf.keras using synthetic data](#)

Using accelerated hardware

- [TensorFlow with GPUs](#)
- [TPUs in Colab](#)

✓ Featured examples

- [Retraining an Image Classifier](#): Build a Keras model on top of a pre-trained image classifier to distinguish flowers.
- [Text Classification](#): Classify IMDB film reviews as either *positive* or *negative*.
- [Style Transfer](#): Use deep learning to transfer style between images.
- [Multilingual Universal Sentence Encoder Q&A](#): Use a machine-learning model to answer questions from the SQuAD dataset.
- [Video Interpolation](#): Predict what happened in a video between the first and the last frame.

Start coding or [generate](#) with AI.

```
# prompt: import pandas as pd
# # Replace this path with your actual path if you're running locally
# file_path = '3d_print_defect_dataset (5).csv'
# # Read the CSV file
# df = pd.read_csv(file_path)
# # Display the first 5 rows of the data
# print(df.head())

import pandas as pd

# Assuming the file is in the same directory as your notebook
# or provide the full path if it is located elsewhere.
try:
    df = pd.read_csv('3d_print_defect_dataset (5).csv')
    print(df.head())
except FileNotFoundError:
    print("Error: '3d_print_defect_dataset (5).csv' not found. Please check the file path.")
```

```
from google.colab import files
uploaded = files.upload() # This will prompt you to upload the file
```



Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving 3d print defect dataset (5) csv to 3d print defect dataset (5) csv

```
import pandas as pd
import io
```

```
# Replace '3d_print_defect_dataset (5).csv' with the actual uploaded filename
df = pd.read_csv(io.BytesIO(uploaded['3d_print_defect_dataset (5).csv']))
print(df.head()) # Display the first few rows
```



	Layer thickness	Nozzle temp	Bed temp	Print speed	Material type	\
0	0.24	256	90	41	Nylon	
1	0.39	260	76	52	PETG	
2	0.40	214	57	34	Nylon	
3	0.28	211	63	86	PLA	
4	0.27	225	90	44	PLA	

	Model complexity	Print time	Printer type	Humidity	Cooling method	\
0	High	4.1	SLS	23	Passive	
1	High	3.8	FDM	59	Fan	
2	High	5.0	SLA	32	Passive	
3	Low	2.6	FDM	26	Fan	
4	Medium	1.8	SLS	22	Passive	

	Defective
0	1

1	1
2	0
3	1
4	0

```
x=df[['Nozzle temp']]
y=df['Defective']
from sklearn.neighbors import KNeighborsClassifier
o=KNeighborsClassifier()
#train the model by using fit
o.fit(x,y)
```



▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier()

```
o.predict([[256]])
```



```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
  array([0])
```



```
o.score(x,y)
```



```
0.644
```

```
x=df[['Layer thickness']]
y=df['Defective']
from sklearn.tree import DecisionTreeClassifier
o=DecisionTreeClassifier()
o.fit(x,y)
```



▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()

```
o.predict([[0.35]])
```



```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
  array([0])
```



```
o.score(x,y)
```

```
0.567
```

```
x=df[['Layer thickness']]
y=df['Defective']
from sklearn.ensemble import RandomForestClassifier
o=RandomForestClassifier()
o.fit(x,y)
o.predict([[0.35]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
  array([0])
```

```
o.score(x,y)
```

```
0.567
```

Start coding or [generate](#) with AI.

```
x=df[['Layer thickness']]
y=df['Defective']
from sklearn.ensemble import GradientBoostingClassifier
o=GradientBoostingClassifier()
o.fit(x,y)
o.predict([[0.35]])
```

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
  array([0])
```

```
o.score(x,y)
```

```
0.567
```

```
x=df[['Layer thickness']]
y=df['Defective']
from sklearn.svm import LinearSVC
o=LinearSVC()
o.fit(x,y)
```



▼ LinearSVC ⓘ ?

`LinearSVC()``o.predict([[0.35]])`

```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
array([0])
```

`o.score(x,y)`

0.644

```
x=df[['Layer thickness']]
y=df['Defective']
from sklearn.linear_model import LogisticRegression
o=LogisticRegression()
o.fit(x,y)
o.predict([[0.35]])
```



```
/usr/local/lib/python3.11/dist-packages/sklearn/utils/validation.py:2739: UserWarning: >
  warnings.warn(
array([0])
```

`o.score(x,y)`

0.522

Start coding or [generate](#) with AI.